Week 4

✅Process Concept

- Diagram of Process State & Process State
  - 5 state process model
  - 7 state process model
- Process Scheduling Queues
- Process Control Block (PCB)
- Context Switch
- Addition of Medium Term Scheduling

Content

✅Process Concept

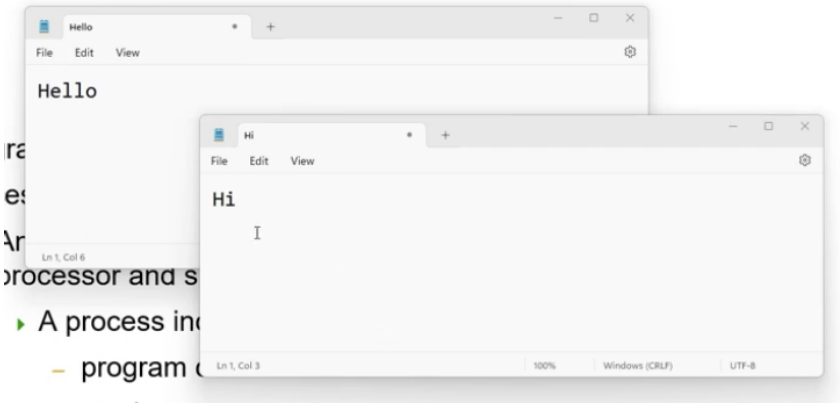
In the previous lecture, we have use the term program when it should be the term concept, in OS world → This 2 terms are different.

- In this, we'll see what the term process is, and we won't use the term program for process anymore.

- Program: An inactive unit, such as a file stored on a disk.
- Process – a program in execution;
  - An active entity, which requires a set of resources, including a processor and special registers, to perform its function
    - A process includes:
      - program counter
      - stack
      - data section
  - A single instance of an executable program
  - Used in Time-Sharing System, also called Task
  - Batch system – jobs

- Program → an inactive unit, such as a file stored on a disk.
  - The file notepad.exe on your hard drive is a program.
- Process → a program in execution. (active unit)
  - When you start the window of notepad.exe program, you are starting a new process.
  - You can start more than 1 process for a program → you can open 2 notepad windows, which mean you start 2 notepad processes.
  - When we say a process is an active unit → it mean a process require a set of resources and have it working status.
  - The info related to its working status is
    1. Program counter → the address of the next instruction that will be executed.
    2. Stack → area of memory to store local variable data and the written address of a function that called another function.
    3. Data Section → memory area to store global data.

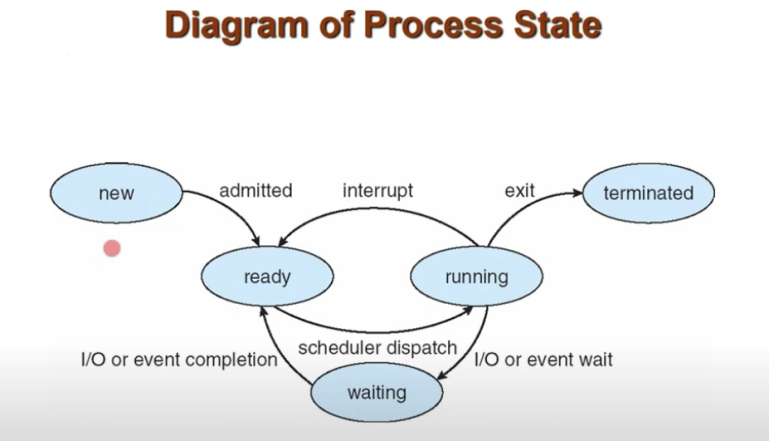
    Each process has it own working information set. For example, we can have 2 notepad processing that can store different data.



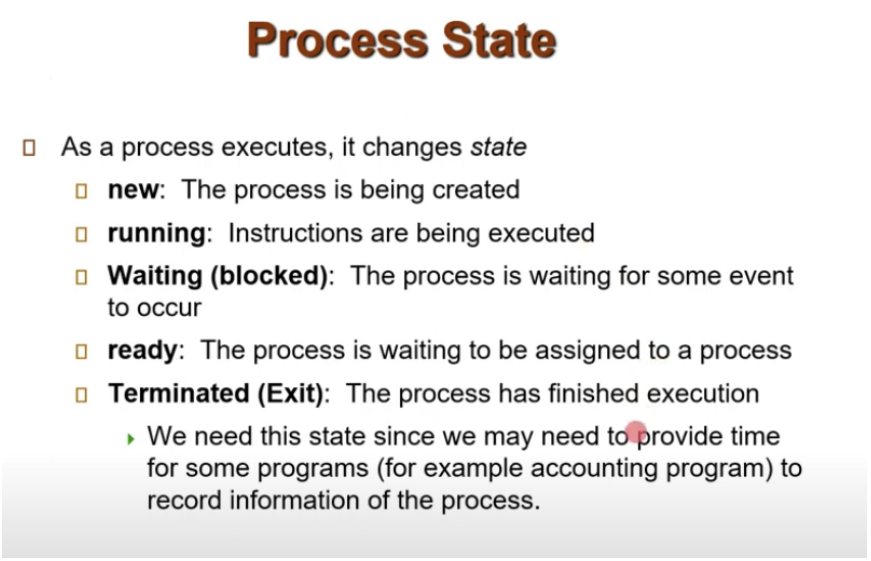
- A process is a single instance of an executable program → like 1 notepad running window is 1 process.
- In a Time Sharing System, a process can be called Task.
  - The term task in Single-user Single-Tasking, ... is a process
  - After this, when we learn thread, we'll know that the term task can be thread as well.
- In Batch System, It can be called Jobs.

📍 Diagram of Process State & Process State

📐 5 state process model



Process will change from state to state during it life cycle.



5 states

If we using multi-processing system, or multi-core cpu, at a time, there maybe more than 1 process running. But we'll explain using system that have only 1 cpu and 1 core.

1. new → when process was first created, it'll be here (state of initialization)

after initialization is done, OS will move the state to ready state

2. ready → the process is now ready to run, wait for OS to choose it to run.
    - There's maybe more than 1 ready process in the system.
    - process wait in queue call "ready queue", and OS choose from here.

Each process that get choose

3. running - the process is now using the cpu time
    - when the process execute the last statement → terminated
    - when some interrupt occur (timer interrupt, time slot expired) → ready (wait for it next term)
    - when need some I/O or wait for some event that may take long time → waiting
        - why not go back to ready → process in ready state must be ready to run (I/O is running slow)
        - go to ready state when I/O or event is completed.

Why process that finish running have to go to terminated state, why not just exist?

- You can think of terminated state as state that OS gonna deallocation the resources from the process → after that the process can terminate.

7 state process model

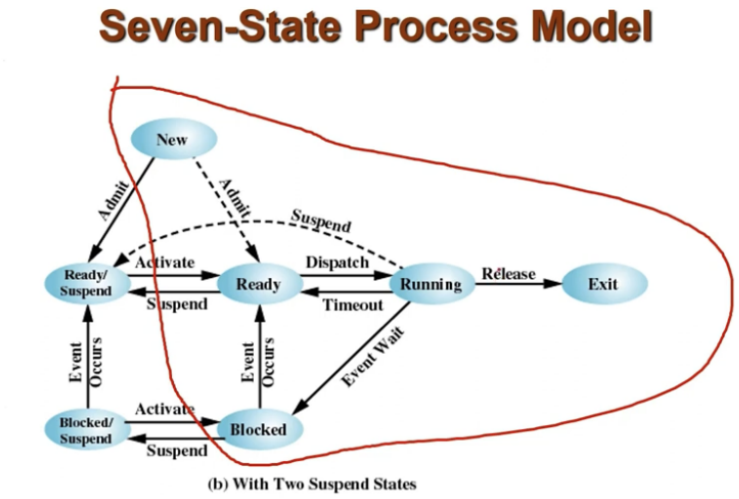


Figure 3.9 Process State Transition Diagram with Suspend States

In some situation, the 5 state model is not enough to model the system.

> When we talk about 5 state model, we are talking about processes that are in computer memory, in the situation that the computer memory is not enough, then we need 2 more state to model the system.

- Inside the red circle is 5 state model that we have discussed previously
- Blocked is the same as waiting in 5 state model.

2 more state to model the system (model processing that are not in memory, that are in the storage(hard drive, SSD)

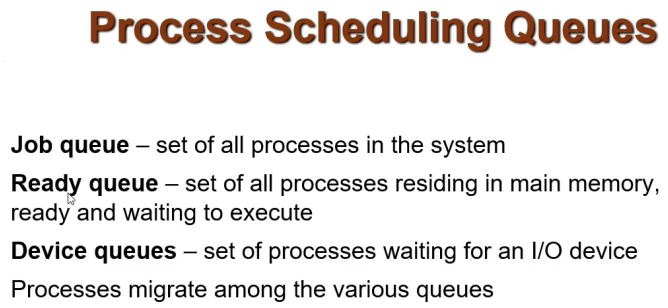
1. Ready/ Suspend
    - OS can move some processes from ready to ready/suspend when the memory is not enough, and can move them back, if the memory is okay.

- The process that is running, when it is interrupted, if the memory is not enough → ready/ suspends

2. Block/ Suspend
    - Block and Block suspends can move back and forth.
    - If event occur(i/o that they are waiting occur)→ move to ready/suspend

---

📍 Process Scheduling Queues

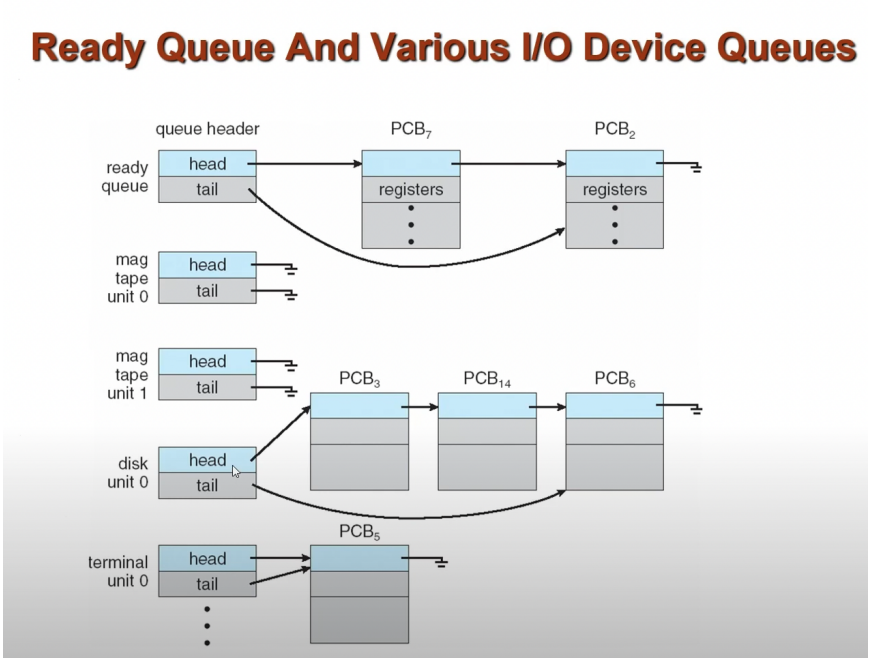


we can start many process, and there's maybe processes that are in the same state and the same time.

So, we need queue for this processes.

- Job queue - queue of the processes that are in new state
- Ready queue - queue of the process that are in the ready state.
- Device queue - queue of the process in the waiting state that wait for I/O device.

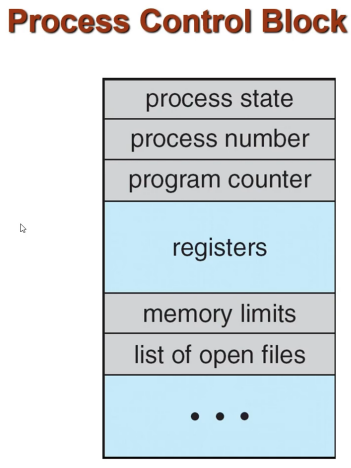
And if we have more than 1 I/O devices, the queue might be something just like this one.



This is queue for the processes that are waiting to use the disk, terminal

Since, OS has to manage all processes in the system, OS must have information about each process, so the idea is the process that created in the system must have PCB (memory area reserve by OS to store the info about each process).

📍 Process Control Block (PCB)



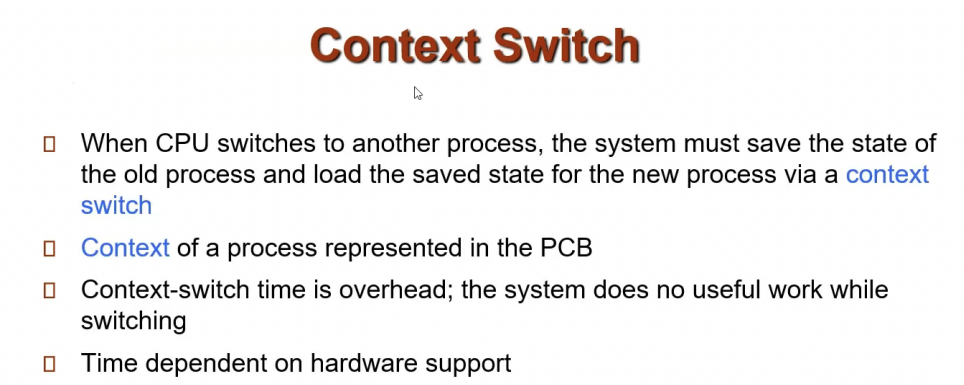
Actually, there are a lot of info that OS need to manage the processes, But the thing we should know is.

1. **Process State**
    - State of process right now

2. **Process Number (Process Id)**
    - Each process must have unique ID, so that OS can refer to the process correctly
3. **Program counter**
    - Store the address of the next instructions that the process will be execute.
4. **Registers**
    - Area to store the value of the cpu register when the process is interrupted, so that when the OS allow this process to run again, the value in this area will be copy back to the register CPU and then the process can continue.
5. Memory Limits
    - Amount of Memory that this process is allowed to use.
6. List of Open files

We'll pay attention to the first 4.

📍 Context Switch



- Context Switch → one important function that OS need to do for the process.
- Context Switch is the process that OS interrupt the running process and allow some other process to run instead
    - For example, if process A is running until its time slot expired, then OS need to stop process A and choose process B from ready queue to run

instead of process A. If time slot of process B expired, ...

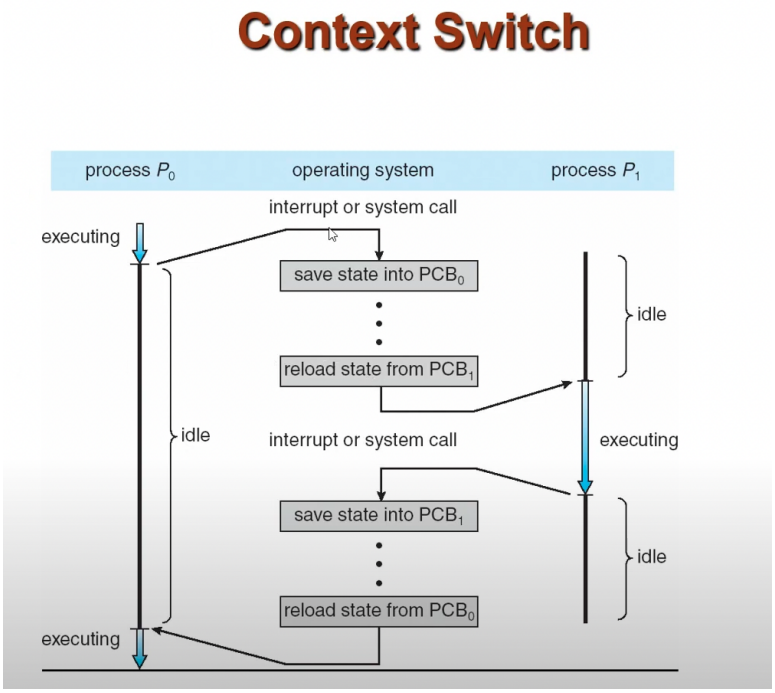
And then why we call this process Context Switch? What does Context mean?

- Actually, recall back to → when interrupt occur, OS need to stop the running process but OS can't just stop, OS has to save the info of the interrupted process of that time. So when OS allows process to run again, it can continue from where it was interrupted.
- The info that OS need to save for the interrupted process is call **"Context".**

Context is store in PCB of each process.

If process A is interrupted, then the context of process A will be store in the PCB of process A.

If OS have to allow process B to run instead of process A → OS need to load the context from the PCB of process B, then allow process B to run. ——→ why this is call context switch.



Assuming that in this system we only have 2 user processes (P0, P1)

- Timeline on the left hand side → Timeline for process P0.

- Timeline on the right hand side → Timeline for process P1.
- In the middle → Timeline for OS

Assuming that there are 2 processes running in the system.

- P0 is running and then interrupt occurs, An OS need to switch to process P1 to run instead of P0. → OS has to save the state or the context of P0 into the PCB of P0.
    - OS does this because when it bring process P0 to run again, process P0 can continue from this interrupted point.
- Before OS can allow process P1 to run, OS need to reload the state of process P1 from PCB of P1, after OS done this one, it allowed process P1 to run.
- Assuming that process P1 get interrupted again, OS has to do the same thing.

> 📎 When OS does context switch, the user processes have to de idle.

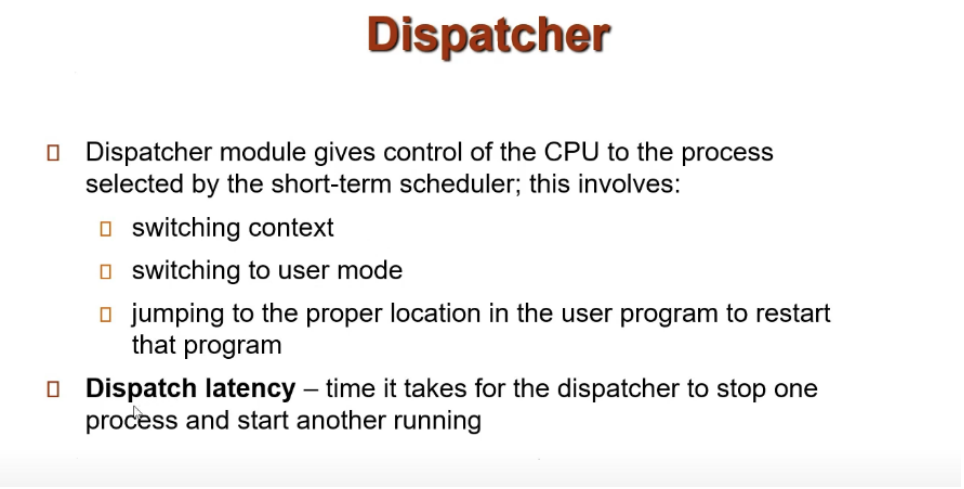
So, the idea is the context-switch time is overhead, but context-switch is required, otherwise we can't do something like time-sharing system. If we don't have context-switch, it mean we have to allow 1 process to run until it finish, before we can start a new process.

In summary, context-switch is required, but it is time overhead

- The way we can solve this problem is
    - We all use should make sure that the context-switch is not occur too often.
    - If the context-switch is needed, we need it to be done as fast as possible.
    - For example, we may use the special hardware that can copy the cpu registers to memory faster.

📍 Dispatcher

(The part of OS that does the context switch)

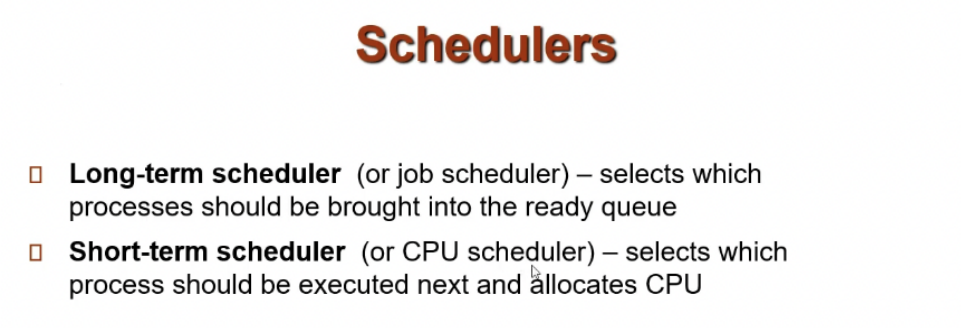


- Dispatch latency - The time to switch from one process to another. The time to stop one process until start a new process.

Dispatcher does context-switch but the part of the CPU that choose what process to run next is schedulers (CPU scheduler)

📍 Schedulers

(Schedulers - part of CPU that choose what process to run next)

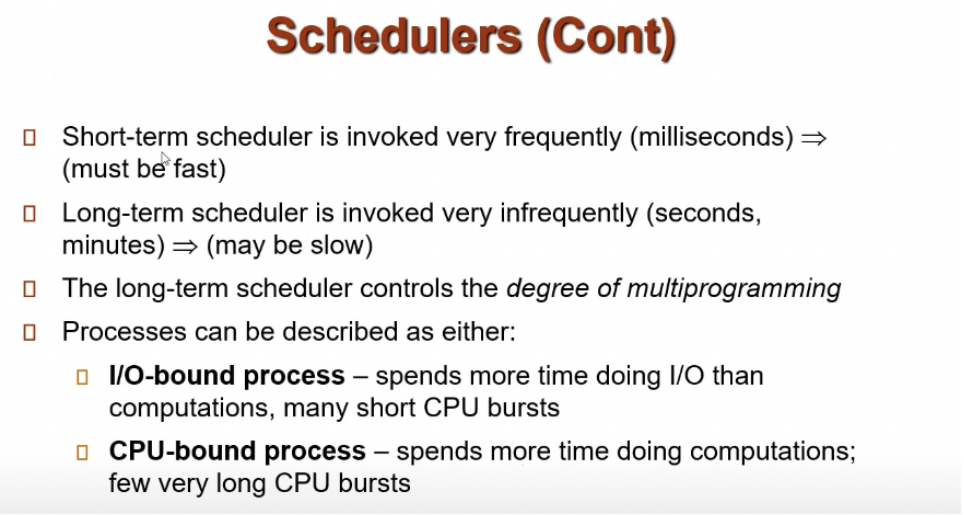


More detail of this will be discussed later

> 👉 **How Dispatcher work with Schedulers?**
>
> - Assume that process A running, and Schedulers said that it's now the time for process B to run, then schedulers will tell Dispatcher that you have to stop process A and bring process B. And Dispatcher does that, stop process A and bring process B to run.

There are 2 major types of schedulers.

1. Long-term scheduler (or job scheduler)
    - Select process from jobs queue to ready queue.
2. Short-term scheduler (or cpu scheduler)
    - Choose process from ready queue to run.



- Short-term scheduler is invoked very very fast.
- Long-term scheduler is invoked very infrequently.
- Long-term scheduler may be more important when we using batch system
    - Because it can control the degree of multiprogramming.
        - What is degree of multiprogramming? - number of program that will be loaded to memory at a time.
    - How long term scheduler controls the degree of multiprogramming?
- There are 2 major type of process.
    - I/O-bound process → always spend time doing I/O, less time on CPU.
    - CPU-bound process → always use CPU, less use I/O.

  So, if the long-term scheduler is not good and it load too many I/O bound process, then the cpu will be idle, and many process will be waiting in device queue. Or if it another way around, I/O will be idle, and many process will be waiting on a queue to use CPU.

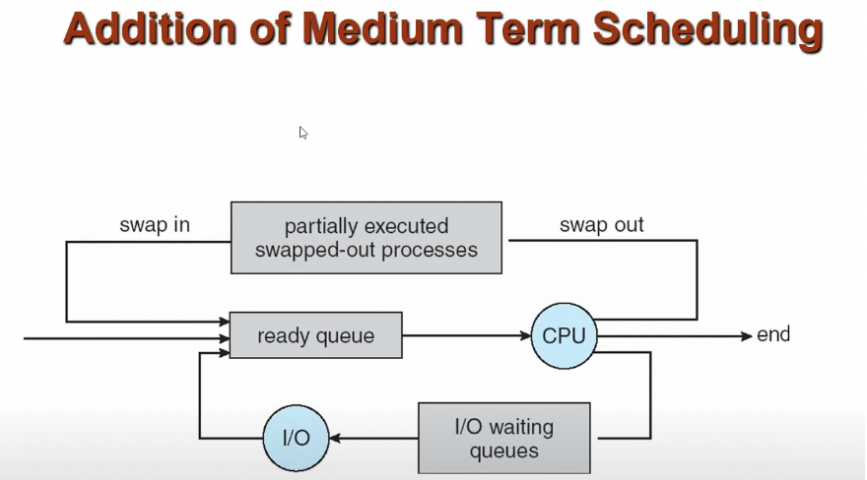
> Good Long-term scheduler should balance between I/O bound process and CPU-bound process.

We might not see the role of long-term scheduler much in the Desktop OS we use today, it have more important role in mainframe OS system. → in our class, we not focus on long term scheduler.

For short-term scheduler, we have a topic based on short-term scheduler.

📍 Addition of Medium Term Scheduling.

Another type of scheduler, lie between short term and long term



The idea is we use to swap the process in and out of memory, bring the process from the waiting state back to ready state.
